FLUTTER SQLITE DATABASE BASICS

LAB SHEET – 7 | All 17 Programs with Separate Code

Setup (do this once): Create a Flutter project using `flutter create sqlite_lab7`. In `pubspec.yaml`, add `sqflite: ^2.4.2` and `path: ^1.9.1` under dependencies, then run `flutter pub get`. For each practical below, paste that program's code into `lib/main.dart` and run the app. Each program is a separate example.

Note: The examples use a local SQLite database on Android/iOS. Some programs insert sample data when the database is first created so that the output can be viewed immediately.

1. Create an SQLite database using sqflite

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final dbPath = await getDatabasesPath();
  final path = p.join(dbPath, 'college.db');
  await openDatabase(path, version: 1);
  runApp(const MaterialApp(home: Home()));
}

class Home extends StatelessWidget {
  const Home({super.key});
  @override
  Widget build(BuildContext context) => const Scaffold(
    body: Center(child: Text('SQLite Database Created Successfully')),
  );
}
```

2. Create a students table

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> createDatabase() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, version) async {
    await db.execute('''
      CREATE TABLE students(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        roll_number TEXT,
        course TEXT
      )
    ''');
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await createDatabase();
  runApp(const MaterialApp(home: Scaffold(
    body: Center(child: Text('Students Table Created Successfully')),
  )));
}
```

3. Insert a student using insert()

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openStudentDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final db = await openStudentDb();
  await db.insert('students', {
    'name': 'Rahul', 'roll_number': '101', 'course': 'BCA'
  });
  runApp(const MaterialApp(home: Scaffold(
    body: Center(child: Text('Student Inserted Successfully')),
  )));
}
```

4. Insert student data entered through TextField widgets

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await openDb();
  runApp(const MaterialApp(home: AddStudentPage()));
}

class AddStudentPage extends StatefulWidget {
  const AddStudentPage({super.key});
  @override
  State<AddStudentPage> createState() => _AddStudentPageState();
}

class _AddStudentPageState extends State<AddStudentPage> {
  final name = TextEditingController();
  final roll = TextEditingController();
  final course = TextEditingController();

  Future<void> save() async {
    final db = await openDb();
    await db.insert('students', {
      'name': name.text.trim(),
      'roll_number': roll.text.trim(),
      'course': course.text.trim(),
    });
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Student Saved')),
    );
    name.clear(); roll.clear(); course.clear();
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Add Student')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(children: [
        TextField(controller: name, decoration: const InputDecoration(labelText: 'Name')),
        TextField(controller: roll, decoration: const InputDecoration(labelText: 'Roll Number')),
        TextField(controller: course, decoration: const InputDecoration(labelText: 'Course')),
        ElevatedButton(onPressed: save, child: const Text('Save Student')),
      ]),
    ),
  );
}
```

5. Insert a student using rawInsert()

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final db = await openDb();
  await db.rawInsert(
    'INSERT INTO students(name, roll_number, course) VALUES(?, ?, ?)',
    ['Aman', '102', 'BCA'],
  );
  runApp(const MaterialApp(home: Scaffold(
    body: Center(child: Text('Record Inserted Using rawInsert()')),
  )));
}
```

6. Display all students using query() and ListView

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Riya','roll_number':'103','course':'BCA'});
    await db.insert('students', {'name':'Aman','roll_number':'104','course':'MCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await openDb();
  runApp(const MaterialApp(home: StudentListPage()));
}

class StudentListPage extends StatefulWidget {
  const StudentListPage({super.key});
  @override
  State<StudentListPage> createState() => _StudentListPageState();
}

class _StudentListPageState extends State<StudentListPage> {
  List<Map<String, Object?>> students = [];
  @override
  void initState() { super.initState(); loadStudents(); }

  Future<void> loadStudents() async {
    final db = await openDb();
    final data = await db.query('students');
    if (mounted) setState(() => students = data);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Student Records')),
    body: ListView.builder(
      itemCount: students.length,
      itemBuilder: (context, i) => ListTile(
        title: Text(students[i]['name'].toString()),
        subtitle: Text('Roll: ${students[i]['roll_number']}  Course: ${students[i]['course']}'),
      ),
    ),
  );
}
```

7. Display students using rawQuery()

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Neha','roll_number':'105','course':'BCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await openDb();
  runApp(const MaterialApp(home: RawQueryPage()));
}

class RawQueryPage extends StatefulWidget {
  const RawQueryPage({super.key});
  @override
  State<RawQueryPage> createState() => _RawQueryPageState();
}

class _RawQueryPageState extends State<RawQueryPage> {
  List<Map<String, Object?>> rows = [];
  @override
  void initState() { super.initState(); load(); }

  Future<void> load() async {
    final db = await openDb();
    final result = await db.rawQuery('SELECT * FROM students');
    if (mounted) setState(() => rows = result);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Students - rawQuery')),
    body: ListView(children: rows.map((s) => ListTile(
      title: Text(s['name'].toString()),
      subtitle: Text('Roll: ${s['roll_number']} | ${s['course']}'),
    )).toList()),
  );
}
```

8. Update an existing student using update()

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Ravi','roll_number':'106','course':'BCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final db = await openDb();
  final records = await db.query('students', where: 'roll_number = ?', whereArgs: ['106']);
  if (records.isNotEmpty) {
    await db.update('students', {'name':'Ravi Kumar','course':'MCA'},
      where: 'roll_number = ?', whereArgs: ['106']);
  }
  runApp(const MaterialApp(home: Scaffold(
    body: Center(child: Text('Student Record Updated')),
  )));
}
```

9. Delete a student using delete()

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Karan','roll_number':'107','course':'BCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final db = await openDb();
  await db.delete('students', where: 'roll_number = ?', whereArgs: ['107']);
  runApp(const MaterialApp(home: Scaffold(
    body: Center(child: Text('Student Deleted Successfully')),
  )));
}
```

10. Delete all students using a Clear All button

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Pooja','roll_number':'108','course':'BCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await openDb();
  runApp(const MaterialApp(home: ClearPage()));
}

class ClearPage extends StatelessWidget {
  const ClearPage({super.key});
  Future<void> clear(BuildContext context) async {
    final db = await openDb();
    await db.delete('students');
    if (context.mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('All student records deleted')),
      );
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Clear Student Records')),
    body: Center(child: ElevatedButton(
      onPressed: () => clear(context),
      child: const Text('Clear All'),
    )),
  );
}
```

11. Search a student by roll number using WHERE

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Simran','roll_number':'109','course':'BCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await openDb();
  runApp(const MaterialApp(home: SearchPage()));
}

class SearchPage extends StatefulWidget {
  const SearchPage({super.key});
  @override
  State<SearchPage> createState() => _SearchPageState();
}

class _SearchPageState extends State<SearchPage> {
  final controller = TextEditingController();
  List<Map<String, Object?>> result = [];

  Future<void> search() async {
    final db = await openDb();
    final data = await db.query('students',
      where: 'roll_number = ?', whereArgs: [controller.text.trim()]);
    setState(() => result = data);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Search Student')),
    body: Column(children: [
      TextField(controller: controller,
        decoration: const InputDecoration(labelText: 'Enter Roll Number')),
      ElevatedButton(onPressed: search, child: const Text('Search')),
      Expanded(child: ListView(children: result.map((s) =>
        ListTile(title: Text(s['name'].toString()),
          subtitle: Text('Course: ${s['course']}'))).toList())),
    ]),
  );
}
```

12. Display students of a selected course using DropdownButton

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Asha','roll_number':'110','course':'BCA'});
    await db.insert('students', {'name':'Mohit','roll_number':'111','course':'MCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await openDb();
  runApp(const MaterialApp(home: CoursePage()));
}

class CoursePage extends StatefulWidget {
  const CoursePage({super.key});
  @override
  State<CoursePage> createState() => _CoursePageState();
}

class _CoursePageState extends State<CoursePage> {
  String course = 'BCA';
  List<Map<String, Object?>> students = [];

  Future<void> load() async {
    final db = await openDb();
    final data = await db.query('students',
      where: 'course = ?', whereArgs: [course]);
    setState(() => students = data);
  }

  @override
  void initState() { super.initState(); load(); }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Students by Course')),
    body: Column(children: [
      DropdownButton<String>(
        value: course,
        items: ['BCA','MCA','BBA'].map((c) =>
          DropdownMenuItem(value: c, child: Text(c))).toList(),
        onChanged: (v) { if (v != null) { setState(() => course = v); load(); } },
      ),
      Expanded(child: ListView(children: students.map((s) =>
        ListTile(title: Text(s['name'].toString()),
          subtitle: Text(s['roll_number'].toString()))).toList())),
    ]),
  );
}
```

13. Display students in ascending order of name using orderBy

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Zoya','roll_number':'112','course':'BCA'});
    await db.insert('students', {'name':'Aman','roll_number':'113','course':'MCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final db = await openDb();
  final students = await db.query('students', orderBy: 'name ASC');
  runApp(MaterialApp(home: Scaffold(
    appBar: AppBar(title: const Text('Name: A to Z')),
    body: ListView(children: students.map((s) =>
      ListTile(title: Text(s['name'].toString()))).toList()),
  )));
}
```

14. Display total number of records in students table

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    await db.insert('students', {'name':'Isha','roll_number':'114','course':'BCA'});
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final db = await openDb();
  final result = await db.rawQuery('SELECT COUNT(*) AS total FROM students');
  final count = Sqflite.firstIntValue(result) ?? 0;
  runApp(MaterialApp(home: Scaffold(
    body: Center(child: Text('Total Student Records: $count',
      style: const TextStyle(fontSize: 22))),
  )));
}
```

15. Student model class with toMap() and fromMap()

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

class Student {
  final int? id;
  final String name, rollNumber, course;

  Student({this.id, required this.name,
    required this.rollNumber, required this.course});

  Map<String, Object?> toMap() => {
    'id': id, 'name': name,
    'roll_number': rollNumber, 'course': course,
  };

  factory Student.fromMap(Map<String, Object?> map) => Student(
    id: map['id'] as int?,
    name: map['name'] as String,
    rollNumber: map['roll_number'] as String,
    course: map['course'] as String,
  );
}

Future<Database> openDb() async {
  final path = p.join(await getDatabasesPath(), 'students.db');
  return openDatabase(path, version: 1, onCreate: (db, v) async {
    await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
  });
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final db = await openDb();
  final student = Student(name: 'Riya', rollNumber: '115', course: 'BCA');
  await db.insert('students', student.toMap());
  final rows = await db.query('students');
  final list = rows.map((row) => Student.fromMap(row)).toList();
  runApp(MaterialApp(home: Scaffold(
    body: ListView(children: list.map((s) =>
      ListTile(title: Text(s.name),
        subtitle: Text('${s.rollNumber} - ${s.course}'))).toList()),
  )));
}
```

16. DatabaseHelper class for open, insert, read, update and delete

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

class DatabaseHelper {
  DatabaseHelper._();
  static final DatabaseHelper instance = DatabaseHelper._();
  static Database? _db;

  Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'helper_demo.db');
    _db = await openDatabase(path, version: 1, onCreate: (db, v) async {
      await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll_number TEXT, course TEXT)');
    });
    return _db!;
  }

  Future<int> insert(Map<String, Object?> data) async =>
    (await database).insert('students', data);

  Future<List<Map<String, Object?>>> read() async =>
    (await database).query('students');

  Future<int> update(int id, Map<String, Object?> data) async =>
    (await database).update('students', data,
      where: 'id = ?', whereArgs: [id]);

  Future<int> delete(int id) async =>
    (await database).delete('students',
      where: 'id = ?', whereArgs: [id]);
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await DatabaseHelper.instance.insert({
    'name':'Kavya', 'roll_number':'116', 'course':'BCA'
  });
  final records = await DatabaseHelper.instance.read();
  runApp(MaterialApp(home: Scaffold(
    body: ListView(children: records.map((s) =>
      ListTile(title: Text(s['name'].toString()))).toList()),
  )));
}
```

17. Student Record Management Application using Student model and DatabaseHelper

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart' as p;

class Student {
  final int? id;
  final String name, roll, course;
  Student({this.id, required this.name, required this.roll, required this.course});
  Map<String, Object?> toMap() =>
    {'id': id, 'name': name, 'roll': roll, 'course': course};
  factory Student.fromMap(Map<String, Object?> m) => Student(
    id: m['id'] as int?, name: m['name'] as String,
    roll: m['roll'] as String, course: m['course'] as String);
}

class DatabaseHelper {
  DatabaseHelper._();
  static final instance = DatabaseHelper._();
  static Database? _db;
  Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'student_manager.db');
    _db = await openDatabase(path, version: 1, onCreate: (db, v) async {
      await db.execute('CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, roll TEXT UNIQUE, course TEXT)');
    });
    return _db!;
  }
  Future<int> insert(Student s) async =>
    (await database).insert('students', s.toMap());
  Future<List<Student>> read() async {
    final rows = await (await database).query('students');
    return rows.map(Student.fromMap).toList();
  }
  Future<int> update(Student s) async =>
    (await database).update('students', s.toMap(),
      where: 'id = ?', whereArgs: [s.id]);
  Future<int> delete(int id) async =>
    (await database).delete('students', where: 'id = ?', whereArgs: [id]);
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await DatabaseHelper.instance.database;
  runApp(const MaterialApp(home: StudentManager()));
}

class StudentManager extends StatefulWidget {
  const StudentManager({super.key});
  @override
  State<StudentManager> createState() => _StudentManagerState();
}

class _StudentManagerState extends State<StudentManager> {
  final helper = DatabaseHelper.instance;
  final name = TextEditingController();
  final roll = TextEditingController();
  final course = TextEditingController();
  List<Student> students = [];
  int? editId;

  @override
  void initState() { super.initState(); load(); }

  Future<void> load() async {
    final data = await helper.read();
    if (mounted) setState(() => students = data);
  }

  Future<void> save() async {
    if (name.text.trim().isEmpty || roll.text.trim().isEmpty ||
        course.text.trim().isEmpty) return;
    final s = Student(id: editId, name: name.text.trim(),
```

17. Student Record Management Application using Student model and DatabaseHelper (continued)

```
      roll: roll.text.trim(), course: course.text.trim());
    try {
      if (editId == null) { await helper.insert(s); }
      else { await helper.update(s); }
      name.clear(); roll.clear(); course.clear();
      setState(() => editId = null);
      await load();
    } catch (e) {
      if (mounted) ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Roll number must be unique')));
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Student Record Management')),
    body: Padding(
      padding: const EdgeInsets.all(12),
      child: Column(children: [
        TextField(controller: name, decoration: const InputDecoration(labelText: 'Name')),
        TextField(controller: roll, decoration: const InputDecoration(labelText: 'Roll Number')),
        TextField(controller: course, decoration: const InputDecoration(labelText: 'Course')),
        ElevatedButton(onPressed: save,
          child: Text(editId == null ? 'Add Student' : 'Update Student')),
        Expanded(child: ListView.builder(
          itemCount: students.length,
          itemBuilder: (context, i) {
            final s = students[i];
```

```
            return ListTile(
              title: Text(s.name),
              subtitle: Text('${s.roll} - ${s.course}'),
              onTap: () {
                setState(() {
                  editId = s.id; name.text = s.name;
                  roll.text = s.roll; course.text = s.course;
                });
              },
              trailing: IconButton(
                icon: const Icon(Icons.delete),
                onPressed: () async { await helper.delete(s.id!); load(); },
              ),
            );
          },
        )),
      ]),
    ),
  );
}
```

How to run each practical

```
flutter create sqlite_lab7
cd sqlite_lab7
# Add sqflite and path in pubspec.yaml
flutter pub get
# Paste ONE program at a time into lib/main.dart
flutter run
```

For GitHub, create a repository (for example, flutter-sqlite-lab-7), add the Flutter project files, and push with:

```
git init
git add .
git commit -m "Solve Flutter SQLite Lab Sheet 7"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/flutter-sqlite-lab-7.git
git push -u origin main
```

Replace YOUR-USERNAME with your GitHub username. Do not upload the build/ directory. If you want to keep all 17 practicals in one repository, save each code in a separate folder or file and copy the selected code to lib/main.dart when running it.